Lab 1 Report:

Data Preparation Techniques for Machine Learning

Name: Ahad Ather

In [219…

```
# Import necessary libraries

%matplotlib inline
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import math
```

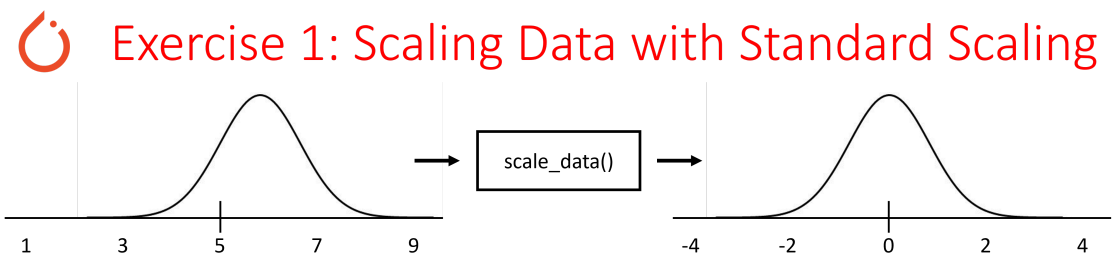
In [2]:

```
from IPython.display import Image # For displaying images in colab jupyter cell
```

In [3]:

```
Image('lab1_exercise1.PNG', width = 1000)
```

Out[3]:



- In Machine Learning, the dataset is usually scaled ahead of time so that it is easier for the computer to **learn** and **understand** the problem.
- One of the most frequently used method is '**standard scaling**', where the data is scaled by $z = (x - \mu)/\sigma$. (x = original datapoint, μ = mean of the data, σ = standard deviation)
- Write a function "**scale_data()**" which takes 2D NumPy array as an input and perform standard scaling on its columns. The function should output a new 2D array containing scaled column data.
- Test your function with selected columns in CMS calorimeter dataset (**hgcal.csv**).
- Plot the scaled dataset for the selected columns by using the provided matplotlib histogram function.

66

In [7]:

```
# Load the dataset (.csv) using pandas package

CMS_calori_dataset = pd.read_csv('hgcal.csv')

# .head directive on the panda dataframe displays the first n-rows

CMS_calori_dataset.head(n = 10)
```

Out[7]:

	Unnamed: 0	x	y	z	eta	phi	energy	trackId
0	0	179.50383	-23.632137	-7.878280	-0.0435	-0.130900	0.200126	462412
1	1	-143.63881	110.217940	-72.706795	-0.3915	2.487094	2.734594	493395
2	2	179.50383	-23.632120	-146.429610	-0.7395	-0.130900	0.423910	1
3	3	-172.67310	54.443620	-238.065340	-1.0875	2.836160	0.713950	493640
4	4	-180.88046	7.897389	-238.065340	-1.0875	3.097959	0.000000	495225
5	5	-180.88045	-7.897438	-238.065340	-1.0875	-3.097959	0.034491	495225
6	6	-152.69838	-97.279590	-265.020540	-1.1745	-2.574361	0.580138	460126
7	7	-23.63213	179.503810	-325.172060	-1.3485	1.701696	0.411487	465028
8	8	-152.69835	97.279594	89.977780	0.4785	2.574361	0.183141	1383
9	9	-176.76110	39.187016	107.930240	0.5655	2.923426	0.337551	4421

In [9]:

```
# Convert the panda dataframe into numpy 2D array

CMS_calori_dataset_np = CMS_calori_dataset.to_numpy()

# The converted numpy array has the dimension of 420 (rows) x 8 (columns)

print(CMS_calori_dataset_np.shape)
```

```
(420, 8)
```

In [146…

```
# Extract only x, y, z, eta, phi and energy columns from the dataset and stack them
# Name this new 2D array CMS_calori_dataset_np_sub.
# The array should have dimension 420 (rows) x 6 (columns)

CMS_calori_dataset_np_sub = CMS_calori_dataset_np[:,1:7]
print(CMS_calori_dataset_np_sub.shape)
```

```
(420, 6)
```

In [156…

```
# Create the scaling function

def scale_data(data):

    x_dim = data.shape[1] # number of columns
    y_dim = data.shape[0] # number of rows
    np.hsplit(data, x_dim) # splits (420, 6) shape matrix array into 6 different ar
    empty_x = [[] for i in range(x_dim)] # create an empty 6 column array

    for i in range(x_dim): # looping through each column i in columns x, y, z, eta,
        data_column = data[:, i] # all rows of i-th column
        means_column = np.mean(data_column) # means of i-th column
        stds_column = np.std(data_column) # standard deviations of i-th column
        for x in range(y_dim): # we look through each x-th row data point by i-th c
            empty_x[i].append((data_column[x]-means_column)/stds_column) # appendin
```

```
    scaled_data = np.stack([empty_x[0], empty_x[1], empty_x[2], empty_x[3], empty_x
    scaled_data = np.transpose(scaled_data)

    return scaled_data
```

In [158…

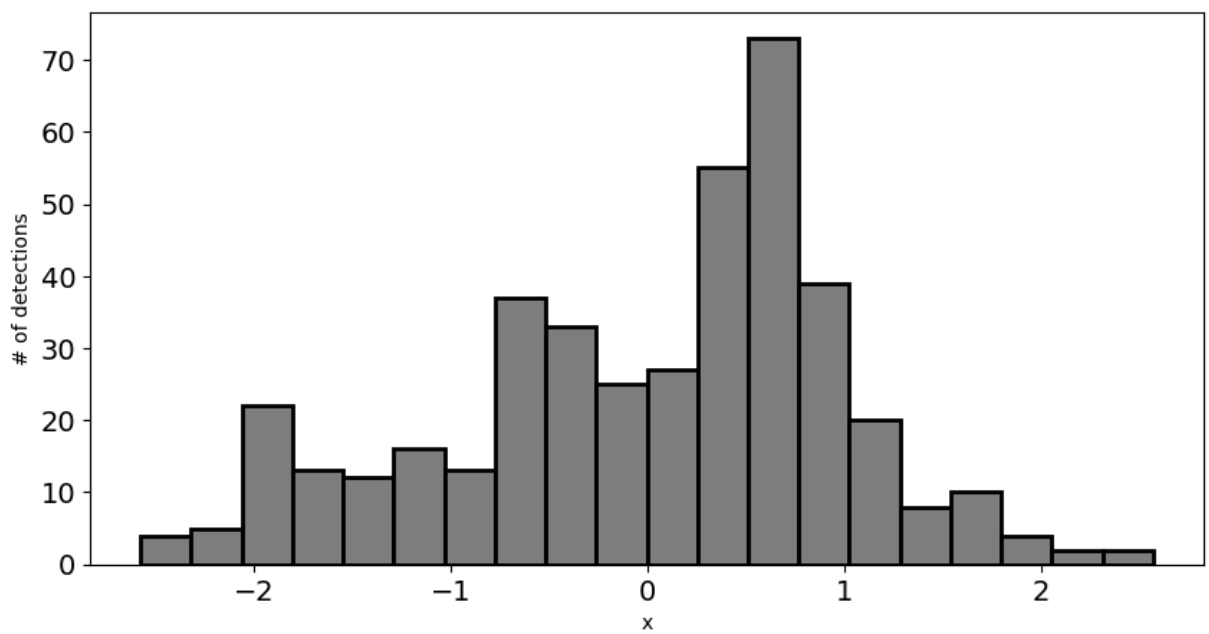
```
# Test the function with CMS_calori_dataset_np_sub

CMS_calori_dataset_np_sub_scaled = scale_data(CMS_calori_dataset_np_sub)
CMS_calori_dataset_np_sub_scaled.shape
```

Out[158…

```
(420, 6)
```

In [164…

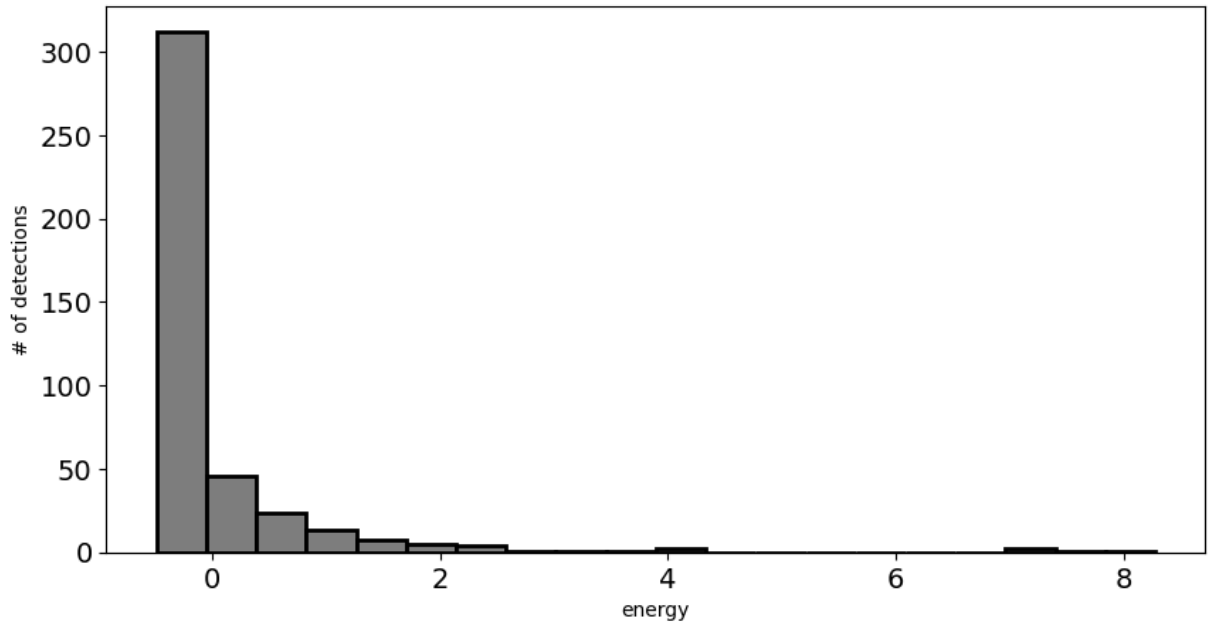
```
# Confirm the data is scaled for 'x' column

plt.figure(figsize = (10, 5))

plt.hist(CMS_calori_dataset_np_sub_scaled[:, 0], bins = 20, facecolor = 'grey', edg
plt.xticks(fontsize=14)
plt.yticks(fontsize=14)

# Add proper x-label and y-label

plt.xlabel("x")
plt.ylabel("# of detections")

plt.show()
```



In [165…

```
# Confirm the data is scaled for 'energy' column

plt.figure(figsize = (10, 5))

plt.hist(CMS_calori_dataset_np_sub_scaled[:, 5], bins = 20, facecolor = 'grey', edg
plt.xticks(fontsize=14)
plt.yticks(fontsize=14)
```

```
# Add proper x-label and y-label

plt.xlabel("energy")
plt.ylabel("# of detections")

plt.show()
```

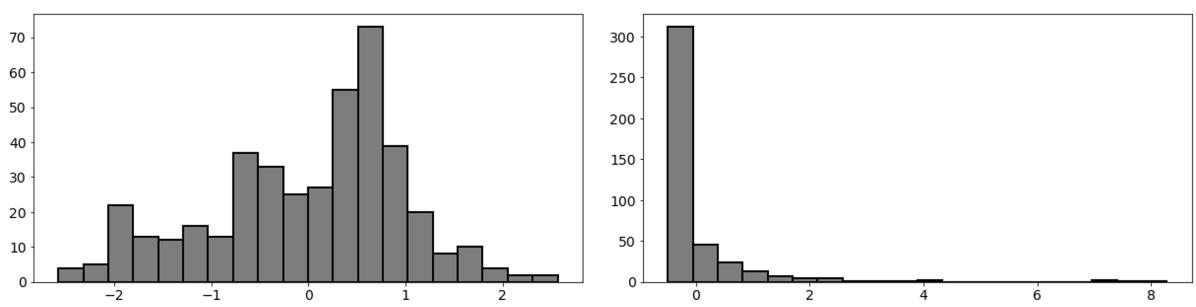


Expected histogram outputs - Feel free to style your plot differently

In [169…

```
Image('lab1_e1_expected_outputs.PNG', width = 1000)
```

Out[169…



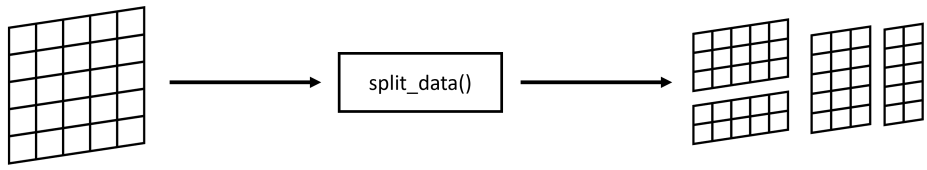
Scaled ‘x’ Scaled ‘energy’

In [171…

```
Image('lab1_exercise2.PNG', width = 1000)
```

Out[171…

Exercise 2: Data Splitting



- In this exercise you will write a function called split_data() which given a NumPy array, it splits the array into sub-arrays.
- Data splitting is used to divide the dataset into training, validation and testing sets, which we will describe in later lab.
- The function should take following parameters
  - arr – 2D NumPy array representing a dataset
  - split_proportions – a list containing split ratios, e.g., [0.2, 0.3, 0.5]
  - axis – a direction to be splitted (0 = row-wise, 1 = column-wise)
- Test your function on the scaled dataset from exercise 1 with given parameters in the lab template.
- Confirm that your sub arrays have correct dimensions by printing their shape

68

In [305…

```
# Create the splitting function

def split_data(arr, split_proportions, axis):
    # Returns a list of numpy sub-arrays according to split proportions

    if axis == 0: # if we want to split row-wise
        splits = [i * arr.shape[0] for i in split_proportions] # multiply each of 3
        splits = [math.ceil(i) for i in splits] # round each decimal data point up
        for i in range(len(splits) - 1): # create i to loop through each index of s
            splits[i+1] = splits[i] + splits[i+1] # transform # of rows per split t

    if axis == 1: # if we want to split column-wise
        splits = [i * arr.shape[1] for i in split_proportions]
        splits = [math.ceil(i) for i in splits]
        for i in range(len(splits) - 1):
            splits[i+1] = splits[i] + splits[i+1]

    split_data_list = np.split(arr, splits, axis = axis)

    return split_data_list
```

In [307…

```
# Test your split function against scaled CMS Calorimieter dataset from exercise 1

sub_data_list_1 = split_data(arr = CMS_calori_dataset_np_sub_scaled,
                                    split_proportions = [0.6, 0.2, 0.2], axis = 0)
```

In [309…

```
# Confirm that dataset has been split into correct shapes
# The correct dimensions should be (252, 6) (84, 6) (84, 6)

print(sub_data_list_1[0].shape, sub_data_list_1[1].shape, sub_data_list_1[2].shape)
```

```
(252, 6) (84, 6) (84, 6)
```

In [311…

```
# Test your split function against scaled CMS Calorimieter dataset from exercise 1
```

```
sub_data_list_2 = split_data(arr = CMS_calori_dataset_np_sub_scaled,
                                                    split_proportions = [0.5, 0.5], axi
```

In [313…

```
# Confirm that dataset has been split into correct shapes
# The correct dimensions should be (420, 3) (420, 3)

print(sub_data_list_2[0].shape, sub_data_list_2[1].shape)
```

```
(420, 3) (420, 3)
```